Daily Challenge: Pinecone Serverless Reranking in Action

Why are we doing this?

Reranking models boost search relevance by assigning similarity scores between a query and documents, then reordering results so the most pertinent information appears first. In contexts like healthcare, this helps clinicians quickly access the most critical clinical notes.

Part 1: Load Documents & Execute Reranking Model

```
!pip install -U -q pinecone==6.0.1 pinecone-notebooks
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 421.4/421.4 kB 8.9 MB/s et
```

```
import os
if not os.environ.get("PINECONE_API_KEY"):
    from pinecone_notebooks.colab import Authenticate
    Authenticate()

from pinecone import Pinecone
api_key = os.environ.get("PINECONE_API_KEY")
pc = Pinecone(api_key=api_key)
```

```
query = "Tell me about Apple's products"
documents = [
    "Apples are round fruits that grow on trees and come in varieties
    "The Apple iPhone 15 features a titanium design and the A17 Pro c
    "Eating an apple a day is a common health tip because they are hi
    "Apple recently released the Vision Pro, a spatial computer that
    "The MacBook Air with the M3 chip is known for being incredibly t
]

reranked = pc.inference.rerank(
    model="bge-reranker-v2-m3",
    query=query,
    documents=[{"id": str(i), "text": doc} for i, doc in enumerate(do
    top_n=3
)

def show_reranked_results(query, matches):
    print(f"Query: {query}\n")
    print("-" * 50)
    for i, m in enumerate(matches):
```

```
        print(f"Rank {i+1} | Score: {m.score:.4f}")
        print(f"Document: {m.document.text}\n")

show_reranked_results(query, reranked.data)
```

```
Query: Tell me about Apple's products

--------------------------------------------------
Rank 1 | Score: 0.2190
Document: Apple recently released the Vision Pro, a spatial computer t

Rank 2 | Score: 0.0532
Document: The Apple iPhone 15 features a titanium design and the A17 P

Rank 3 | Score: 0.0385
Document: The MacBook Air with the M3 chip is known for being incredib
```

Part 2: Setup a Serverless Index for Medical Notes

```
!pip install -q pandas torch transformers
```

```
import os
import time
import pandas as pd
from pinecone import Pinecone, ServerlessSpec
from transformers import AutoTokenizer, AutoModel
import torch

cloud = os.getenv('PINECONE_CLOUD', 'aws')
region = os.getenv('PINECONE_REGION', 'us-east-1')

spec = ServerlessSpec(cloud=cloud, region=region)

index_name = 'medical-notes-index'
```

```
if pc.has_index(name=index_name):
    pc.delete_index(name=index_name)

pc.create_index(
    name=index_name,
    dimension=384,
    metric='cosine',
    spec=spec
)

while not pc.describe_index(index_name).status['ready']:
    time.sleep(1)

print(f"Index '{index_name}' is ready!")
```

```
Index 'medical-notes-index' is ready!
```

Part 3: Load the Sample Data

```
import requests
import tempfile
import os
import pandas as pd

with tempfile.TemporaryDirectory() as tmpdirname:
    file_path = os.path.join(tmpdirname, "sample_notes_data.jsonl")

    url = "https://raw.githubusercontent.com/pinecone-io/examples/mai

    response = requests.get(url)
    response.raise_for_status()

    with open(file_path, "wb") as f:
        f.write(response.content)

    df = pd.read_json(file_path, orient='records', lines=True)

print("Data shape:", df.shape)
display(df.head())
```

```
Data shape: (100, 3)
```

	id	values	metadata
0	P011	[-0.2027486265, 0.2769146562, -0.1509393603, 0...	{'advice': 'rest, hydrate', 'symptoms': 'heada...
1	P001	[0.1842793673, 0.4459365904, -0.0770567134, 0....	{'tests': 'EKG, stress test', 'symptoms': 'che...
2	P002	[-0.2040648609, -0.1739618927, -0.2897160649, ...	{'HbA1c': '7.2', 'condition': 'diabetes', 'med...
3	P003	[0.1889383644, 0.2924542725, -0.2335938066, -0...	{'symptoms': 'cough, wheezing', 'diagnosis': '...
4	P004	[-0.12171068040000001, 0.1674752235, -0.231888...	{'referral': 'dermatology', 'condition': 'susp...

Part 4: Upsert Data into the Index

```
index = pc.Index(name=index_name)

index.upsert_from_dataframe(df)

def is_fresh(index):
    stats = index.describe_index_stats()
    vector_count = stats.total_vector_count
    print(f"Current vector count in index: {vector_count}")
    return vector_count > 0
```

```
while not is_fresh(index):
    print("Waiting for vectors to be indexed...")
    time.sleep(5)

print("\nIndex ready!")
display(index.describe_index_stats())
```

```
sending upsert requests: 100%                    100/100 [00:01<00:00, 70.54it/s]
Current vector count in index: 100

Index ready!
{'dimension': 384,
 'index_fullness': 0.0,
 'metric': 'cosine',
 'namespaces': {'': {'vector_count': 100}},
 'total_vector_count': 100,
 'vector_type': 'dense'}
```

Part 5: Query & Embedding Function

```
def get_embedding(input_question):
    model_name = 'sentence-transformers/all-MiniLM-L6-v2'
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModel.from_pretrained(model_name)

    encoded_input = tokenizer(input_question, padding=True, truncatio

    with torch.no_grad():
        model_output = model(**encoded_input)
        embeddings = model_output.last_hidden_state[0].mean(dim=0) #
        embeddings = model_output.last_hidden_state.mean(dim=1)

    return embeddings[0].tolist()
```

```
question = "What are the common treatments for acute chest pain?
query_vector = get_embedding(question)

print(f"Vector length: {len(query_vector)}")

if len(query_vector) == 384:
    results = index.query(vector=query_vector, top_k=10, include
    sorted_matches = sorted(results['matches'], key=lambda x: x[

    for i, match in enumerate(sorted_matches[:3]):
        print(f"Match {i+1} (Score: {match['score']:.4f}):")

        metadata = match.get('metadata', {})

        note_content = metadata.get('text') or metadata.get('con

        print(f"Note Content: {note_content[:150]}...\n")
else:
    print("Error: Vector dimension is still not 384.")
```

```
Loading weights: 100%  103/103 [00:00<00:00, 440.89it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED   :can be ignored when loading from different task/archite
Vector length: 384
Match 1 (Score: 0.5221):
Note Content: {'symptoms': 'chest pain', 'tests': 'EKG, stress test'}.

Match 2 (Score: 0.4175):
Note Content: {'advice': 'over-the-counter pain relief, stretching', '

Match 3 (Score: 0.3629):
Note Content: {'diagnosis': 'bronchitis', 'symptoms': 'cough, wheezing
```

Part 6: Display & Rerank Clinical Notes

```
def show_results(question, matches):
    print(f"Question: '{question}'")
    print("\nInitial Search Results (Bi-Encoder):")
    for i, match in enumerate(matches):
        print(f"{str(i+1).rjust(4)}. ID: {match['id']}")
        print(f"      Score: {match['score']:.4f}")
        metadata = match.get('metadata', {})
        text_content = metadata.get('text') or metadata.get('con
        print(f"      Note: {text_content[:100]}...")
        print('')

show_results(question, sorted_matches)

transformed_documents = [
    {
        'id': match['id'],
        'reranking_field': '; '.join([f"{key}: {value}" for key,
    }
    for match in results['matches']
]

refined_query = "What are the specific clinical protocols and me

reranked_results = pc.inference.rerank(
    model="bge-reranker-v2-m3",
    query=refined_query,
    documents=transformed_documents,
    rank_fields=["reranking_field"],
    top_n=3,
    return_documents=True,
)
```

```
)


def show_reranked_results(question, matches):
    print(f"Refined Question: '{question}'")
    print("\n--- FINAL RERANKED RESULTS (Cross-Encoder) ---")
    for i, match in enumerate(matches):
        print(f"{str(i+1).rjust(4)}. ID: {match.document.id}")
        print(f"      New Rerank Score: {match.score:.4f}")
        print(f"      Clinical Data: {match.document.reranking_f

show_reranked_results(refined_query, reranked_results.data)
```

```
Question: 'What are the common treatments for acute chest pain?'

Initial Search Results (Bi-Encoder):
   1. ID: P001
      Score: 0.5221
      Note: {'symptoms': 'chest pain', 'tests': 'EKG, stress test'}..

   2. ID: P0100
      Score: 0.4175
      Note: {'advice': 'over-the-counter pain relief, stretching', 's

   3. ID: P003
      Score: 0.3629
      Note: {'diagnosis': 'bronchitis', 'symptoms': 'cough, wheezing'

   4. ID: P016
      Score: 0.3591
      Note: {'condition': 'heart murmur', 'referral': 'cardiology'}..

   5. ID: P059
      Score: 0.3424
      Note: {'symptoms': 'joint pain', 'treatment': 'NSAIDs, rest'}..

   6. ID: P030
      Score: 0.3363
      Note: {'symptoms': 'panic attacks', 'treatment': 'benzodiazepin

   7. ID: P092
      Score: 0.3331
      Note: {'condition': 'dehydration', 'treatment': 'IV fluids'}...

   8. ID: P044
      Score: 0.3331
      Note: {'condition': 'dehydration', 'treatment': 'IV fluids'}...

   9. ID: P063
      Score: 0.3314
      Note: {'diagnosis': 'pneumonia', 'symptoms': 'cough, fever', 't

  10. ID: P095
      Score: 0.3246
      Note: {'symptoms': 'back pain', 'treatment': 'physical therapy'

Refined Question: 'What are the specific clinical protocols and medic

--- FINAL RERANKED RESULTS (Cross-Encoder) ---
```

1. ID: P001
   New Rerank Score: 0.0572
   Clinical Data: symptoms: chest pain; tests: EKG, stress test...

2. ID: P059
   New Rerank Score: 0.0056
   Clinical Data: symptoms: joint pain; treatment: NSAIDs, rest...

3. ID: P030
   New Rerank Score: 0.0038
   Clinical Data: symptoms: panic attacks; treatment: benzodiazepi